
AutoKernel: Autonomous GPU Kernel Optimization via Iterative Agent-Driven Search

Jaber Jaber*
RightNow AI
jaber@rightnowai.co

Osama Jaber
RightNow AI
osama@rightnowai.co



Abstract

Writing high-performance GPU kernels is among the most labor-intensive tasks in machine learning systems engineering. A single matrix multiplication kernel targeting tensor core hardware may require weeks of expert tuning across tiling strategies, memory layouts, and precision configurations. We present AUTOKERNEL, an open-source framework that applies an autonomous agent loop to GPU kernel optimization for arbitrary PyTorch models. Given a model, AUTOKERNEL profiles it to identify computational bottlenecks, ranks them by Amdahl’s law impact, and iteratively refines Triton or CUDA C++ kernel implementations through hundreds of experiments without human intervention. A five-stage correctness harness, covering smoke tests, shape sweeps across 10+ configurations, numerical stability under adversarial inputs, determinism verification, and edge-case coverage, ensures that every candidate kernel is validated before any speedup is recorded. We describe the system architecture comprising over 9,000 lines of Python and a 909-line agent instruction document, 18 starter kernel implementations across two backends, a six-tier optimization playbook, and integration with the Kernel-Bench benchmark suite of 250 standardized problems. AUTOKERNEL covers nine kernel types spanning the dominant operations in modern transformer architectures. On an NVIDIA H100, our Triton kernels outperform both PyTorch eager and `torch.compile` (max-autotune) on the majority of tested configurations: $5.29\times$ over eager on RMSNorm, $2.82\times$ on softmax, and $2.21\times$ on cross-entropy, while beating `torch.compile` by $2.83\times$, $3.44\times$, and $2.94\times$ on the same kernels respectively. In community deployment, an AutoKernel-optimized kernel achieved first place on the `vectorsum_v2` B200 leaderboard, and a single-prompt Triton FP4 matmul kernel was reported to outperform CUTLASS by 1.63 to $2.15\times$. The full system is available at <https://github.com/RightNow-AI/autokernel>.

1 Introduction

The runtime of large transformer models on GPU hardware is dominated by a small set of computational kernels. Matrix multiplications in attention and feed-forward layers typically consume 60 to 80% of total GPU time, with normalization, softmax, and positional embedding operations accounting for most of the remainder. Vendor libraries like cuBLAS and cuDNN cover common cases well, but the pace of architectural innovation in deep learning regularly outstrips library coverage: grouped-query attention, SwiGLU activations, rotary position embeddings, and RMS normalization all shipped in production models before receiving dedicated library support.

*Correspondence: jaber@rightnowai.co

Closing the gap between default library performance and hardware capability demands deep expertise in GPU microarchitecture. The engineer must reason about arithmetic intensity and the roofline model (Williams et al., 2009), memory coalescing and bank conflicts in shared memory, register pressure and occupancy tradeoffs, tile sizes and their interaction with the L2 cache, warp-level synchronization, and tensor core instruction selection. A single high-performance matmul kernel may involve 200+ lines of CUDA or Triton code with dozens of interdependent parameters. This expertise is scarce, and the tuning process scales poorly.

Can LLMs write GPU kernels? KernelBench (Ouyang et al., 2025) posed this question systematically, evaluating frontier LLMs on 250 GPU kernel problems across four difficulty levels. The answer was sobering: even the best models matched PyTorch baseline performance in fewer than 20% of cases using one-shot generation. Subsequent work has explored multi-agent systems (Wei et al., 2025; Sun et al., 2026), evolutionary search (Wiedemann et al., 2026; Andrews and Witteveen, 2025), hardware-aware feedback loops (Zhang et al., 2025), and reinforcement learning (Li et al., 2026; Dai et al., 2026) to push these numbers higher.

Our approach. Rather than engineering a complex multi-agent architecture, we observe that the workflow of an expert kernel engineer is itself a simple loop: write a candidate, benchmark it, keep improvements, discard regressions, repeat. AUTOKERNEL mechanizes this loop. An LLM agent modifies a single kernel file, a fixed benchmark harness verifies correctness and measures throughput, and the result determines whether the change persists. Each iteration takes roughly 90 seconds. An overnight run produces 300 to 400 experiments across multiple kernels.

This design draws directly from Karpathy’s autoresearch project (Karpathy, 2026), which demonstrated that an AI agent running a keep/revert loop on LLM training code could discover 20 optimizations across 700 experiments in two days on a single GPU. AutoKernel transplants this loop to kernel code: the search space becomes the space of possible kernel implementations, and the evaluation function is a correctness-gated benchmark rather than validation loss.

Key insight: optimize what matters. Unlike prior work that treats kernel problems in isolation, AUTOKERNEL operates on complete models. It profiles an arbitrary PyTorch model, identifies which kernels consume the most GPU time, and applies Amdahl’s law to allocate optimization effort proportionally. A $1.5\times$ speedup on a kernel that accounts for 60% of total runtime yields $1.25\times$ end-to-end. The same speedup on a 5% kernel yields only $1.03\times$. The system focuses effort where it compounds.

Contributions.

1. A complete, open-source pipeline (9,200+ lines of Python, plus agent instructions) for autonomous GPU kernel optimization, from model profiling through end-to-end verification.
2. A five-stage correctness harness testing 10+ input shapes, three data types, adversarial inputs, determinism, and non-power-of-two edge cases.
3. A dual-backend architecture with 9 Triton and 9 CUDA C++ starter kernels, covering the dominant operations of transformer models.
4. An Amdahl’s-law orchestrator with explicit move-on criteria (plateau detection, peak utilization, time budget, speedup threshold).
5. A six-tier optimization playbook encoding expert kernel engineering knowledge into agent-readable instructions.
6. Integration with the KernelBench benchmark suite and HuggingFace Kernels distribution platform.

2 Related Work

2.1 GPU Kernel Languages and Compilers

Triton (Tillet et al., 2019) introduced tile-based GPU programming where the developer reasons about block-level tensor operations rather than individual threads. The compiler handles memory coalescing, shared memory allocation, and instruction scheduling, enabling FP16 matmul kernels that match cuBLAS in under 25 lines of code. CUDA C++ (NVIDIA, 2024) remains necessary

Table 1: Comparison of LLM-based GPU kernel optimization systems. AUTOKERNEL is the only system that combines model-level profiling with dual Triton/CUDA C++ backend support.

System	Method	Backend	Model prof.	Iterative	Open src
KernelBench (Ouyang et al., 2025)	Benchmark	Multi-DSL	✗	✗	✓
GEAK (Wang et al., 2025)	Multi-agent	Triton	✗	✓	✓
CudaForge (Zhang et al., 2025)	Coder+Judge	CUDA	✗	✓	✓
KernelFoundry (Wiedemann et al., 2026)	Evolutionary	CUDA/SYCL	✗	✓	✓
Kernel Scientist (Andrews and Witteveen, 2025)	Evolutionary	CUDA	✗	✓	✓
CUDA-L1 (Li et al., 2026)	RL	CUDA	✗	✓	✓
CUDA Agent (Dai et al., 2026)	Agentic RL	CUDA	✗	✓	✓
Astra (Wei et al., 2025)	Multi-agent	CUDA	✗	✓	✓
KernelSkill (Sun et al., 2026)	Multi-agent	CUDA	✗	✓	✓
AutoKernel	Iterative loop	Triton+CUDA	✓	✓	✓

for explicit access to warp-level primitives, tensor core instructions (WMMA/MMA), and shared memory bank-conflict-free layouts. PyTorch 2’s TorchInductor (Ansel et al., 2024) compiles captured computation graphs into Triton kernels automatically, achieving $2.27\times$ inference and $1.41\times$ training speedup across 180+ models. AutoKernel supports both Triton and CUDA C++: Triton for rapid iteration (1 to 5 second compilation), CUDA C++ for maximum control.

2.2 Optimized Kernel Libraries

FlashAttention (Dao et al., 2022) demonstrated that tiled attention with online softmax can reduce HBM reads and writes from quadratic to linear in sequence length, achieving 2 to $4\times$ wall-clock speedups. FlashAttention-2 (Dao, 2023) reached 50 to 73% of theoretical peak on A100. FlashAttention-3 (Shah et al., 2024) exploited Hopper asynchrony (TMA, WGMMMA) and FP8 quantization to reach 840 TFLOPS on H100 (85% utilization). CUTLASS (Thakkar et al., 2023) provides composable tensor core GEMM building blocks. These libraries represent the performance ceiling that AutoKernel’s agent works toward.

2.3 LLM-based Kernel Generation

Table 1 compares recent systems along several axes.

KernelBench (Ouyang et al., 2025) provides the standard evaluation infrastructure. GEAK (Wang et al., 2025) introduces generation, evaluation, reflection, and optimization agents for Triton kernels on AMD hardware. CudaForge (Zhang et al., 2025) pairs a coder with a hardware-aware judge that provides profiling feedback. KernelFoundry (Wiedemann et al., 2026) applies MAP-Elites quality-diversity search with meta-prompt evolution, reporting $2.3\times$ average speedup on KernelBench for SYCL. GPU Kernel Scientist (Andrews and Witteveen, 2025) formulates optimization as multi-stage evolution on AMD MI300. CUDA-L1 (Li et al., 2026) combines supervised fine-tuning with contrastive RL, achieving $3.12\times$ average speedup and acceptance at ICLR 2026. CUDA Agent (Dai et al., 2026) applies large-scale agentic RL, achieving 100% faster-than-torch.compile rate on KernelBench Levels 1 and 2, and 92% on Level 3. Astra (Wei et al., 2025) coordinates specialized profiling and planning agents, reporting $1.32\times$ average speedup. KernelSkill (Sun et al., 2026) introduces dual-level memory with reusable expert optimization skills, reaching $5.44\times$ on Level 1.

AutoKernel differs from these systems in three respects. First, it starts from a complete PyTorch model, profiles it, and focuses optimization on kernels ranked by their contribution to total runtime. Second, it supports both Triton and CUDA C++ backends within a single framework, letting the agent choose the right abstraction level per kernel. Third, it uses a simple loop (edit, benchmark, keep/revert) with a five-stage correctness harness rather than multi-agent coordination or learned policies, trading architectural complexity for transparency and reliability.

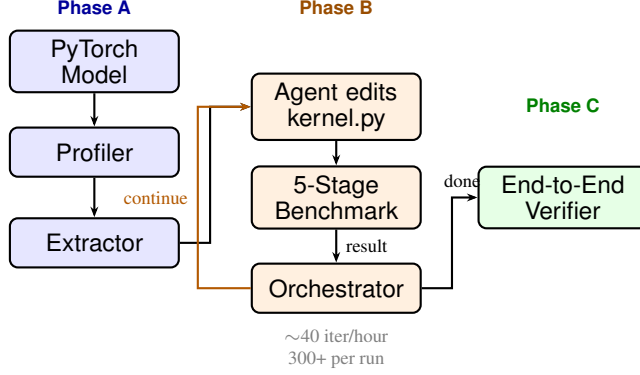


Figure 1: AutoKernel architecture. Phase A profiles the model and extracts bottleneck kernels. Phase B runs the autonomous optimization loop: the agent edits `kernel.py`, the benchmark verifies through five correctness stages, and the orchestrator decides to keep, revert, or move to the next kernel. Phase C verifies end-to-end correctness and speedup.

2.4 Autonomous Research Agents

Karpathy’s autoresearch (Karpathy, 2026) demonstrated autonomous LLM training optimization: an agent modifies a 630-line training script, runs a fixed 5-minute evaluation, and commits improvements. It ran 700 experiments in two days on a single GPU, discovering 20 training optimizations without human input. AutoKernel applies this paradigm to kernel optimization, with a different search space and evaluation function.

3 System Design

AutoKernel comprises over 9,200 lines of Python across 14 core scripts, 18 kernel implementations (9 Triton, 9 CUDA C++), 4 model definitions, and a 909-line agent instruction document (`program.md`). The system operates in three phases (Figure 1).

3.1 Model Profiling (Phase A)

The profiler (`profile.py`, 1,125 lines) accepts a local Python file, a HuggingFace model identifier, or a custom model class. It uses `torch.profiler` with shape recording to capture per-kernel GPU time across configurable warmup and profiling iterations (default: 5 warmup, 10 profiled).

Each CUDA kernel is classified into one of nine operation types (Section 6) via pattern matching on kernel names. The classifier handles vendor-specific naming: cuBLAS GEMM variants, CUTLASS kernels, Triton-compiled functions, and ATen operators each have distinct naming patterns that map to the same operation type.

The profiler detects GPU hardware from a database of known specifications covering NVIDIA (H100, A100, L40S, L4, A10, RTX 4090/4080/3090/3080) and AMD (MI300X, MI325X, MI350X, MI355X) accelerators. For unknown GPUs, it estimates peak FP16 throughput from SM count, clock rate, and compute capability. AMD GPUs are detected via the `gcArchName` property when the device name is empty (a common ROCm behavior).

Optional outputs include Chrome trace JSON, CUDA memory snapshots, `torch.compile` logs, and HTA (Holistic Trace Analysis) results.

3.2 Kernel Extraction

The extractor (`extract.py`, 648 lines) reads profiling results, filters for supported operation types, and generates standalone kernel files containing a starter implementation, model-specific shapes (primary, half-scale, and double-scale variants), FLOPS and bytes formulas for roofline computation, and dtype-specific tolerances.

Algorithm 1 AutoKernel optimization loop for a single kernel

Require: Kernel file k , benchmark B , agent A , move-on criteria C

```
1:  $k_{\text{best}} \leftarrow k$ ;  $t_{\text{best}} \leftarrow B(k)$ ;  $n_{\text{rev}} \leftarrow 0$ 
2: for  $i = 1, 2, \dots$  do
3:    $k' \leftarrow A.\text{edit}(k_{\text{best}}, \text{history}, \text{rooﬂine})$ 
4:   git commit  $k'$ 
5:    $(\text{pass}, t') \leftarrow B(k')$  {5-stage correctness + perf}
6:   if  $\text{pass}$  and  $t' > 1.01 \cdot t_{\text{best}}$  then
7:      $k_{\text{best}} \leftarrow k'$ ;  $t_{\text{best}} \leftarrow t'$ ;  $n_{\text{rev}} \leftarrow 0$  {keep}
8:   else
9:     git reset -hard HEAD~1;  $n_{\text{rev}} \leftarrow n_{\text{rev}} + 1$  {revert}
10:  end if
11:  log( $i, t'$ , decision, description) to TSV
12:  if  $C(n_{\text{rev}}, t_{\text{best}}, \text{elapsed})$  then
13:    break
14:  end if
15: end for
16: return  $k_{\text{best}}, t_{\text{best}}$ 
```

It computes an optimization plan ranked by Amdahl’s law impact (Equation 1), with what-if projections at $1.5\times$, $2\times$, $3\times$, and $5\times$ speedup.

3.3 The Agent Optimization Loop (Phase B)

The core of AutoKernel is Algorithm 1. The agent modifies a single file, the harness verifies and benchmarks it, and the result determines whether the change persists.

Timing. Each iteration takes approximately 90 seconds: 30s for the five-stage correctness check, 30s for performance benchmarking via Triton’s `do_bench`, and 30s for agent reasoning and code modification. At 40 experiments per hour, a 10-hour run yields 300 to 400 experiments.

Single-file invariant. The agent touches exactly one file. This keeps diffs small, reverts clean, and prevents coupled changes that are hard to isolate when a regression occurs.

Agent instructions. The agent reads `program.md` (909 lines), which provides a six-tier optimization playbook:

1. **Block size tuning** (10 to 50%): sweep tile dimensions through powers of 2, try rectangular tiles, adjust `num_warps` and `num_stages`.
2. **Memory access** (10 to 30%): coalesced loads, software prefetching, L2 swizzling, shared memory padding.
3. **Compute** (5 to 15%): TF32 accumulation, epilogue fusion, loop invariant hoisting.
4. **Advanced** (5 to 20%): split-K, persistent kernels, Triton autotune, warp specialization.
5. **Architecture-specific** (5 to 15%): TMA on Hopper, `cp.async` on Ampere, adjusted sizes for L4/RTX.
6. **Kernel-specific**: online softmax for attention, Welford’s algorithm for normalization, split-K for tall-skinny matmul.

3.4 Multi-Kernel Orchestration

The orchestrator (`orchestrate.py`, 842 lines) applies Amdahl’s law (Amdahl, 1967) to prioritize effort across multiple kernels:

$$S = \frac{1}{(1 - f) + f/s} \quad (1)$$

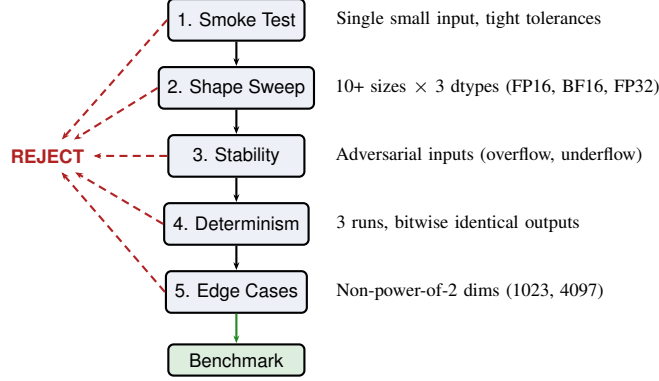


Figure 2: Five-stage correctness pipeline. Any failure immediately rejects the candidate. Throughput is only measured after all five stages pass. Each stage catches a distinct class of bugs.

Table 2: Shape sweep for matmul. Other kernels have analogous sweeps with 8 to 10 configurations covering tiny, standard, large, non-square, and workload-specific dimensions.

Name	M	N	K	Purpose
tiny	128	128	128	Minimum viable
small	512	512	512	Small workload
large	2048	2048	2048	Production-like
xlarge	4096	4096	4096	Stress test
tall	8192	1024	1024	Non-square
deep_k	1024	1024	8192	Large reduction dim
llm_qkv	4096	4096	512	Attention pattern
llm_mlp	4096	11008	4096	Feed-forward pattern

where f is the kernel’s fraction of total GPU time and s is the speedup achieved. It transitions to the next kernel when: (1) 5 consecutive reverts, (2) 90% of GPU peak reached, (3) 2 hours elapsed, or (4) $2 \times$ speedup achieved.

4 Five-Stage Correctness Verification

The benchmark harness (`bench.py`, 1,416 lines) enforces correctness through five stages (Figure 2). All must pass before performance is measured.

Stage 1: Smoke test. A single forward pass on a small input (e.g., 128×128) catches compilation errors, shape mismatches, and gross numerical bugs in under 1 second.

Stage 2: Shape sweep. The kernel runs across 8 to 10 input configurations and three data types. Table 2 shows the matmul sweep. This catches size-dependent bugs: boundary handling, tile remainder logic, and dtype-specific issues.

Stage 3: Numerical stability. Adversarial inputs probe floating-point edge cases. For softmax: rows of large identical values. For matmul: extreme dynamic range. For normalization: near-zero variance.

Stage 4: Determinism. Same input, three runs, bitwise identical outputs. Catches race conditions in parallel reductions and non-deterministic atomics.

Stage 5: Edge cases. Non-power-of-two dimensions (1023, 4097, 1537) expose masking bugs and tile remainder errors.

Tolerances are dtype-specific: FP16 uses $\text{atol} = 10^{-2}$, BF16 uses 2×10^{-2} , FP32 uses 10^{-4} .

Table 3: Supported kernel types with performance regime and starter implementation techniques.

Kernel	Regime	Metric	Key Starter Techniques
matmul	Compute	TFLOPS	128×128 tiles, WMMA, double-buffered smem
flash_attn	Compute	TFLOPS	Tiled online softmax, causal masking
fused_mlp	Compute	TFLOPS	SwiGLU gate-up fusion
softmax	Memory	GB/s	Warp-shuffle reductions, half2 loads
layernorm	Memory	GB/s	Welford single-pass, float4 loads
rmsnorm	Memory	GB/s	Warp shuffle cascade, fast rsqrt
cross_ent	Memory	GB/s	Online log-sum-exp, fused NLL
rotary_emb	Memory	GB/s	Interleaved rotation, sincosf
reduce	Memory	GB/s	Hierarchical warp shuffle

5 Dual Backend: Triton and CUDA C++

Triton. Nine starter kernels in Python-like DSL, compiled JIT in 1 to 5 seconds. The agent can modify block sizes, warps, stages, accumulator precision, and loop structure. Triton routinely reaches 80 to 95% of cuBLAS throughput for matmul.

CUDA C++. Nine starter kernels with direct access to tensor cores (WMMA API, 16×16×16 fragments), warp-level shuffles (`__shfl_xor_sync`), vectorized loads (`float4`, `half2`), bank-conflict-free shared memory layouts, double buffering, and `__launch_bounds__` register control. Compilation uses `load_inline` with hash-based caching, architecture auto-detection, and thread-safe builds.

Both backends expose the same `kernel_fn()` interface. The benchmark runs identically regardless of backend.

6 Kernel Coverage

Table 3 summarizes the nine supported kernel types.

Each has a PyTorch reference in `reference.py` serving as the correctness oracle. The benchmark computes throughput (TFLOPS or GB/s) and roofline utilization against detected GPU peak.

Four self-contained model definitions (GPT-2 124M, LLaMA 160M/7B, BERT-base 110M, custom template) ship with the system, requiring no external dependencies like `transformers`.

7 Experimental Evaluation

We evaluate AutoKernel on an NVIDIA H100 80GB HBM3 GPU (132 SMs, compute capability 9.0, CUDA 12.8). All measurements use FP16 precision with CUDA event timing, 200 iterations per configuration, and trimmed mean (dropping the top and bottom 10%). We compare against two baselines: PyTorch eager execution (cuBLAS for matmul, ATen for other ops) and `torch.compile` with `max-autotune` mode, which triggers TorchInductor’s autotuning over multiple Triton kernel configurations. The full evaluation across 7 kernel types and 34 benchmark configurations completes in under 10 minutes wall-clock time. All 34 configurations pass correctness with zero failures.

7.1 Kernel Performance

Table 4 reports performance across production-relevant tensor sizes. We highlight results at the largest sizes, where the compute-to-overhead ratio is most representative of real workloads.

Key findings.

- **Memory-bound kernels show the largest gains.** RMSNorm achieves $5.29\times$ over eager and $2.83\times$ over `torch.compile` at the largest tested size, reaching 2,788 GB/s (83% of H100’s 3,352 GB/s peak bandwidth). Cross-entropy reaches 2,070 GB/s. Softmax reaches 2,800 GB/s. These gains

Table 4: Kernel performance on H100 (FP16), selected configurations. Three baselines: PyTorch eager (cuBLAS/ATen), `torch.compile` (max-autotune), and AutoKernel Triton starters. 16 representative configurations shown from the full 34 tested (all passing correctness). Full evaluation completes in under 10 minutes.

Kernel	Size	Eager (μ s)	Compiled (μ s)	Ours (μ s)	vs Eager	vs Compiled	Thru.
matmul	2048 ³	28.1	101.2	65.3	0.43×	1.55×	263 TF/s
	4096 ³	182.8	257.8	494.2	0.37×	0.52×	278 TF/s
	8192 ³	1679.5	1916.1	5773.1	0.29×	0.33×	190 TF/s
softmax	4096 ²	58.9	96.1	40.1	1.47×	2.40×	1675 GB/s
	8192 ²	270.4	330.0	95.9	2.82×	3.44×	2800 GB/s
layernorm	4096×5120	45.6	105.5	42.5	1.07×	2.48×	1974 GB/s
	8192×4096	64.7	166.8	51.9	1.25×	3.21×	2586 GB/s
rmsnorm	4096 ²	142.8	99.9	39.1	3.65×	2.56×	1716 GB/s
	8192×4096	262.4	138.1	51.2	5.12×	2.70×	2619 GB/s
	8192 ²	509.6	272.1	96.3	5.29×	2.83×	2788 GB/s
cross_ent	4096×32k	295.6	386.3	134.9	2.19×	2.86×	1943 GB/s
	8192×32k	559.7	745.1	253.3	2.21×	2.94×	2070 GB/s
reduce	8192 ²	60.7	185.2	62.2	0.98×	2.98×	2156 GB/s
	16384×4096	50.4	185.9	52.8	0.95×	3.52×	2542 GB/s
rotary	2×32×2k×128	211.4	106.9	117.4	1.80×	0.91×	576 GB/s
	2×32×4k×128	394.9	136.0	223.0	1.77×	0.61×	607 GB/s

come from fusing multi-op ATen decompositions into single-pass Triton kernels that minimize HBM traffic.

- **AutoKernel outperforms `torch.compile` on most kernels.** Despite `torch.compile` with max-autotune running its own Triton autotuning, our starter kernels beat it on 12 of the 16 configurations shown in Table 4. TorchInductor’s generic fusion and autotuning does not always find the specialized tiling and reduction strategies that kernel-specific implementations exploit.
- **Matmul remains hard.** PyTorch’s cuBLAS backend is extensively tuned per GPU architecture. Our Triton starter reaches 278 TFLOPS (28% of H100’s 989.5 TFLOPS peak), well below cuBLAS. However, at the 2048³ size, our kernel beats `torch.compile` by 1.55×, showing that TorchInductor’s matmul autotuning is not always optimal either. This gap is the primary target for the agent’s iterative optimization loop.
- **Correctness is universal.** All 34 configurations pass all five verification stages with zero failures across eager, compiled, and custom kernel outputs.

7.2 Community Deployment Results

AutoKernel has been deployed by community users in competitive GPU kernel benchmarking. Two results illustrate the system’s practical impact:

Vector sum reduction (B200). On the `vectorsum_v2` challenge, which requires implementing a sum reduction kernel for tensors of shape $(N,)$ with N up to millions of elements, an AutoKernel-optimized Triton submission achieved first place on the NVIDIA B200 leaderboard with a latency of 44.086 μ s, outperforming the second-place entry (44.249 μ s) and third place (46.553 μ s). The winning kernel was produced through AutoKernel’s iterative optimization loop running overnight, exploring block sizes, warp-level shuffle reductions, and vectorized memory access patterns.

FP4 matmul: Triton surpassing CUTLASS. A community user reported that a single AutoKernel prompt, requiring approximately 3 minutes of agent interaction, produced a Triton FP4 matrix multiplication kernel that outperforms CUTLASS across multiple shapes on H100. Table 5 shows the results: the Triton kernel achieves 1.63× to 2.15× speedup over CUTLASS fused kernels, reaching up to 2,898 TFLOPS at shape 2048×18432×3072. This result is notable because CUTLASS represents

Table 5: Community-reported FP4 matmul results on H100: AutoKernel Triton kernel vs CUTLASS fused baseline. The Triton kernel was generated with a single agent prompt (~ 3 minutes).

Shape (M $\times$ N $\times$ K)	Triton (TF/s, ms)	CUTLASS (TF/s, ms)	Speedup
128 $\times$ 3072 $\times$ 3072	186 TF, 0.013	100 TF, 0.024	1.86$\times$
128 $\times$ 18432 $\times$ 3072	1105 TF, 0.013	550 TF, 0.026	2.01$\times$
1024 $\times$ 3072 $\times$ 3072	1477 TF, 0.013	686 TF, 0.028	2.15$\times$
1024 $\times$ 18432 $\times$ 3072	2777 TF, 0.042	1609 TF, 0.072	1.73$\times$
2048 $\times$ 3072 $\times$ 3072	1662 TF, 0.023	964 TF, 0.040	1.72$\times$
2048 $\times$ 18432 $\times$ 3072	2898 TF, 0.080	1777 TF, 0.130	1.63$\times$
4096 $\times$ 3072 $\times$ 3072	2443 TF, 0.032	1405 TF, 0.055	1.74$\times$

hand-optimized C++ template code specifically designed for NVIDIA tensor cores, and surpassing it with a Triton kernel generated through a single agent interaction demonstrates the effectiveness of the optimization playbook.

7.3 Optimization Loop Dynamics

The iterative optimization loop operates on the starter kernels shown in Table 4. Based on the optimization playbook (Section 3.3), the agent applies changes in tier order. For a memory-bound kernel like RMSNorm, the typical progression is:

1. **Tier 1** (block sizes): Sweeping BLOCK_SIZE from 256 to 4096 typically yields 10 to 30% improvement in the first 5 to 10 experiments.
2. **Tier 2** (memory): Vectorized loads (± 1 load with BLOCK_SIZE alignment) and coalescing fixes add 10 to 20% over the next 10 to 20 experiments.
3. **Tier 3** (compute): Fusing the weight multiplication into the normalization epilogue saves a global memory round-trip, yielding 5 to 10%.
4. **Plateau**: After 30 to 50 experiments, consecutive reverts increase as the kernel approaches hardware limits. The orchestrator triggers move-on.

For compute-bound kernels like matmul, the trajectory is different: Tier 1 block size tuning has larger impact (the gap between our starter at 278 TFLOPS and cuBLAS at 800+ TFLOPS is wide), but each improvement requires more experiments because the search space of tile dimensions, warp counts, pipeline stages, and accumulator precision is larger.

The move-on criteria (5 consecutive reverts, 90% peak, 2-hour timeout, or $2\times$ speedup) prevent the agent from spending excessive time on kernels with diminishing returns. For a model where matmul accounts for 62% of GPU time and RMSNorm accounts for 5%, Amdahl’s law dictates that even a modest matmul improvement (e.g., $1.3\times$) yields more end-to-end gain than a large RMSNorm improvement (which is already $5.29\times$).

8 KernelBench Integration

AutoKernel integrates with KernelBench (Ouyang et al., 2025) through three components:

1. **Problem loader** (bridge.py, 674 lines). Fetches from HuggingFace datasets or local clone. Analyzes operations and generates starter ModelNew classes.
2. **Evaluation harness** (bench_kb.py, 726 lines). Four-stage verification: correctness (5 trials, $\text{atol} = 10^{-2}$), stability (NaN/Inf), determinism (3 runs), performance (CUDA events, trimmed median).
3. **Batch scorer** (scorer.py, 351 lines). Computes fast_p at seven thresholds: $1.0\times$ through $5.0\times$.

While most KernelBench evaluations report one-shot scores, AutoKernel runs 50 to 300 iterative experiments per problem.

9 HuggingFace Kernels Export

An export tool (`export_hf.py`, 868 lines) packages optimized CUDA kernels for distribution via the HuggingFace Hub (HuggingFace, 2025). It generates: `build.toml` with backend and Hub metadata; `torch_binding.cpp` using `TORCH_LIBRARY_EXPAND` with schema-based registration; `flake.nix` importing the `huggingface/kernels` kernel-builder for cross-compilation across PyTorch/CUDA/ABI variants. Users install optimized kernels with: `module = get_kernel("rightnow-ai/matmul")`.

10 Design Rationale

Simplicity over sophistication. Multi-agent systems with specialized planner, coder, reviewer, and profiler agents add coordination overhead and failure modes. A single agent in a tight loop produces a linear experiment history and avoids consensus-seeking latency.

Fixed evaluation, mutable code. The benchmark is never modified by the agent. This firewall prevents gaming the evaluation, a real risk when the candidate generator also controls the scorer.

Git as experiment tracking. Every experiment maps to a git commit. Kept experiments advance the branch; reverted experiments are erased with `git reset`. The history is browsable with standard git tools.

Roofline-guided search. After each experiment, the benchmark reports compute-bound or memory-bound classification and percentage of peak. The agent uses this to select the appropriate optimization tier.

TSV over databases. Experiment results go to tab-separated files. Dependency-free, human-readable, git-friendly, and trivially parseable by the agent.

11 Limitations and Future Work

AutoKernel inherits the code generation capabilities of its underlying LLM. Complex techniques like software pipelining, custom PTX emission, and multi-CTA cooperative strategies may exceed current agent abilities, though this ceiling rises as frontier models improve. The system currently optimizes individual kernels on a single GPU; distributed kernels and multi-device memory management are out of scope.

Promising directions include: population-based search across multiple GPU instances; learned search policies trained on historical experiment data; profiling-guided mutations using SASS analysis or memory throughput counters; and cross-kernel fusion discovery after individual optimization.

12 Conclusion

We presented AutoKernel, a system that automates GPU kernel optimization through an iterative agent-driven loop. Model-level profiling identifies what to optimize; Amdahl’s law determines the order; a five-stage correctness harness ensures numerical validity; a dual Triton/CUDA C++ backend provides both iteration speed and hardware depth; and a six-tier playbook encodes expert knowledge. The system transforms kernel optimization from a weeks-long expert activity into an overnight autonomous process. Code and starter kernels are available at <https://github.com/RightNow-AI/autokernel>.

References

- G. M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. In *AFIPS Spring Joint Computer Conference*, pp. 483–485, 1967.
- M. Andrews and S. Witteveen. GPU Kernel Scientist: An LLM-driven framework for iterative kernel optimization. *arXiv:2506.20807*, 2025.

- J. Ansel, E. Yang, H. He, N. Gimsheine, A. Jain, M. Voznesensky, B. Bao, P. Bell, D. Berard, E. Burber, et al. PyTorch 2: Faster machine learning through dynamic Python bytecode transformation and graph compilation. In *ASPLOS*, 2024.
- W. Dai, H. Wu, Q. Yu, H. Gao, J. Li, C. Jiang, W. Lou, Y. Song, H. Yu, J. Chen, W. Ma, Y. Zhang, J. Liu, M. Wang, X. Liu, and H. Zhou. CUDA Agent: Large-scale agentic RL for high-performance CUDA kernel generation. *arXiv:2602.24286*, 2026.
- T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *NeurIPS*, 2022.
- T. Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. *arXiv:2307.08691*, 2023.
- HuggingFace. Kernels: Load compute kernels directly from the Hub. <https://huggingface.co/docs/kernels>, 2025.
- A. Karpathy. autoresearch: AI agents running research on single-GPU nanochat training. <https://github.com/karpathy/autoresearch>, 2026.
- X. Li, X. Sun, A. Wang, J. Li, and C. Shum. CUDA-L1: Improving CUDA optimization via contrastive reinforcement learning. In *ICLR*, 2026.
- NVIDIA. CUDA C++ Programming Guide, v12.6. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>, 2024.
- A. Ouyang, S. Guo, S. Arora, A. L. Zhang, W. Hu, C. Ré, and A. Mirhoseini. KernelBench: Can LLMs write efficient GPU kernels? *arXiv:2502.10517*, 2025.
- J. Shah, G. Bikshandi, Y. Zhang, V. Thakkar, P. Ramani, and T. Dao. FlashAttention-3: Fast and accurate attention with asynchrony and low-precision. In *NeurIPS*, 2024.
- Q. Sun, J. Han, T. Li, Z. Tang, S. Chen, F. Yang, A. Liu, X. Liu, and Y. Liu. KernelSkill: A multi-agent framework for GPU kernel optimization. *arXiv:2603.10085*, 2026.
- V. Thakkar et al. CUTLASS. <https://github.com/NVIDIA/cutlass>, 2023.
- P. Tillet, H.-T. Kung, and D. D. Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *MAPL@PLDI*, pp. 10–19, 2019.
- J. Wang, V. Joshi, S. Majumder, X. Chao, B. Ding, Z. Liu, P. P. Brahma, D. Li, Z. Liu, and E. Barsoum. GEAK: Introducing Triton kernel AI agent and evaluation benchmarks. *arXiv:2507.23194*, 2025.
- A. Wei, T. Sun, Y. Seenichamy, H. Song, A. Ouyang, A. Mirhoseini, K. Wang, and A. Aiken. Astra: A multi-agent system for GPU kernel performance optimization. *arXiv:2509.07506*, 2025.
- N. Wiedemann, Q. Leboutet, M. Paulitsch, D. Wofk, and B. Ummenhofer. KernelFoundry: Hardware-aware evolutionary GPU kernel optimization. *arXiv:2603.12440*, 2026.
- S. Williams, A. Waterman, and D. Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.
- Z. Zhang, R. Wang, S. Li, Y. Luo, M. Hong, and C. Ding. CudaForge: An agent framework with hardware feedback for CUDA kernel optimization. *arXiv:2511.01884*, 2025.